

BANCO DE DADOS RELACIONAL

Stored Procedure

Professora:

Lucineide Pimenta

Recapitulando a aula anterior



- ❑ O que é uma VIEW?
 - ❑ Consulta salva
 - ❑ Usada para facilitar leitura e relatórios
- ❑ Se a VIEW salva uma consulta...
- ❑ Será que podemos salvar uma ação no banco?

BANCO DE DADOS RELACIONAL

Stored Procedure

Objetivos da aula



Conteúdo

O que é Stored Procedure

Diferença SP x Function

Criar uma Stored Procedure

Executar com parâmetros

Aplicar no banco de exemplo

Aplicar no ABP

Objetivo

Entender para quê serve

Saber quando usar cada

Sintaxe e execução

Entrada e lógica

Biblioteca

clima_alerta — INPE/limnologia

Conceitos rápidos

O que é uma Stored Procedure?

Uma Stored Procedure é um bloco de código SQL armazenado no banco, que pode ser executado sempre que necessário.

Ela serve para:

- > automatizar tarefas repetitivas
- > organizar e centralizar a lógica de negócio no banco
- > garantir integridade das regras
- > melhorar performance (menos ida e volta ao BD)
- > permitir execução com um comando simples: CALL

Analogia:

Sem Stored Procedure = toda vez você escreve tudo do zero

Com Stored Procedure = você tem uma “função” salva e só chama ela

VIEW = consulta salva

PROCEDURE = ação salva

Diferença: Stored Procedure x Function

Stored Procedure	Function
Executa ações (inserir, atualizar, deletar)	Calcula e retorna valores
Pode não retornar nada	Sempre retorna algo
Usa CALL	Usa SELECT function()
Ideal para regras de negócio	Ideal para cálculos e validações

Exemplo (Banco Biblioteca)

-- Cadastrar livro automaticamente

CREATE OR REPLACE PROCEDURE

inserir_livro(

 p_titulo VARCHAR,

 p_ano INT,

 p_id_autor INT,

 p_id_editora INT,

 p_paginas INT

)

LANGUAGE plpgsql

AS \$\$

BEGIN

 INSERT INTO livro(titulo,
 ano_publicacao, id_autor, id_editora,
 num_paginas)

 VALUES (p_titulo, p_ano, p_id_autor,
 p_id_editora, p_paginas);

END;

\$\$;

Exemplo (Banco Biblioteca)

❑ Executar

CALL inserir_livro('Dom Casmurro', 1899, 2, 2, 300);

CALL inserir_livro('Memórias Póstumas', 1881, 2, 2, 240);

❑ Verificar

*SELECT * FROM livro;*

Exemplo (Banco Biblioteca)

Procedures (1)

{ } inserir_livro(IN p_titulo charac

1..3 Sequences

Tables

Trigger Functions

Types

Views (3)

Subscriptions

Alerts

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

```
385 $$;  
386  
387 CALL inserir_livro('Dom Casmurro', 1899, 2, 2, 300);  
388 CALL inserir_livro('Memórias Póstumas', 1881, 2, 2, 240);  
389 Select * from livro;  
390
```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 6

✎

📄

 Page No: 1 of 1

⏪

⏴

⏵

⏩

	id_livro [PK] integer ✎	titulo character varying (150) ✎	ano_publicacao integer ✎	id_autor integer ✎	id_editora integer ✎	num_paginas integer ✎
4	4	O Hobbit	1937	1	1	336
5	5	Dom Casmurro	1899	2	2	300
6	6	Memórias Póstumas	1881	2	2	240

Exemplo (Banco Biblioteca)

-- Atualizar número de páginas

CREATE OR REPLACE PROCEDURE

atualizar_paginas(

p_id_livro INT,

p_paginas INT

)

LANGUAGE plpgsql

AS \$\$

BEGIN

UPDATE livro

SET num_paginas = p_paginas

WHERE id_livro = p_id_livro;

END;

\$\$;

Exemplo (Banco Biblioteca)

-- Excluir livro

```
CREATE OR REPLACE PROCEDURE excluir_livro(  
    p_id_livro INT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    DELETE FROM livro  
    WHERE id_livro = p_id_livro;  
END;  
$$;
```

Exemplo (Banco Biblioteca)

Procedures (3)

atualizar_paginas(IN p_id_livro

excluir_livro(IN p_id_livro integ

inserir_livro(IN p_titulo caract

1..3 Sequences

Tables

Trigger Functions

Types

Views (3)

subscriptions

```

405 -- Excluir livro
406 CREATE OR REPLACE PROCEDURE excluir_livro(
407     p_id_livro INT
408 )
409 LANGUAGE plpgsql
410 AS $$

```

Data Output

Messages

Notifications

CREATE PROCEDURE

Query returned successfully in 53 msec.

Validação - Importante

Exemplo com regra

```
CREATE OR REPLACE PROCEDURE  
inserir_livro_validado(  
    p_titulo VARCHAR,  
    p_paginas INT  
)
```

***Aqui estamos colocando regra de negócio
dentro do banco.***

```
LANGUAGE plpgsql  
AS $$  
BEGIN  
    IF p_paginas <= 0 THEN  
        RAISE EXCEPTION 'Número de páginas  
inválido!';  
    END IF;  
  
    INSERT INTO livro(titulo, num_paginas)  
VALUES (p_titulo, p_paginas);  
END;  
$$;
```

Qual a vantagem de usar SP?

Benefício

Performance

Segurança

Reutilização

Consistência

Atomicidade

Automação

Melhor organização

Por quê importa?

Processa dentro do BD: menos tráfego

Pode restringir acesso direto às tabelas

Regras e operações complexas em único ponto

Mesma regra sempre aplicada

Garantem que operações de banco sejam seguras

Operações repetitivas ficam automáticas

Banco mais profissional e gerenciável

Stored Procedures transformam o banco em um agente ativo do sistema, não apenas um repositório de dados.

Desvantagens / limitações

Desvantagem

Manutenção mais difícil
Pode gerar dependência do SGBD
Versão e controle de código mais complexo
Excesso de lógica no BD pode confundir
Debug mais limitado

Impacto

Código espalhado entre backend e BD
Difícil migrar entre PostgreSQL / MySQL / etc
Requer disciplina com Git
SP deve complementar, não substituir Backend
SP não é tão fácil de testar quanto código TS/JS

*SPs são poderosas e se usado demais, vira bagunça.
Se usado certo, vira diferencial profissional.*

Quando usar Views?

Cenário	Usar SP?	Motivo
Rotinas repetitivas	Sim	Evita reescrever código
Rotinas críticas	Sim	Segurança e atomicidade
Regras de negócios sensíveis	Sim	Garantir integridade
Muita lógica no dado	Sim	Calcula dentro do banco, mais rápido
CRUD simples	Não	Pode ser excesso
Lógica de UI / Client	Não	Deve ficar no frontend
Validações complexas de app	Depende	Depende: às vezes melhor no backend

SP é usada quando a regra é do dado, não da tela.

Prática

Nome claro e padrão

Receber parâmetros explícitos

Documentar dentro da SP

Validar parâmetros

Usar transações quando necessário

Evitar lógica de negócios excessiva

Manter simples e modular

Versão e histórico de alterações

Explicação

sp_inserir_usuario, sp_registrar_medicao

Evita depender de estado externo

Comentários explicando o propósito

Evita dados incorretos (ex.: valores negativos)

BEGIN; ... COMMIT; em operações críticas

Use SP para regras de dados, não regras de tela

Uma SP — um propósito

Criar padrão de documentação para alterações

Perguntas frequentes

- ❑ **Professora, posso fazer tudo no backend e ignorar SP?**
Pode, mas vai perder **segurança, performance e consistência**.
Bancos modernos não são caixa de dados — eles processam dados.
- ❑ **Por que o PostgreSQL usa CALL, e não SELECT?**
CALL = procedimento (ação)
SELECT = função (retorna valor)
No PostgreSQL moderno, a sintaxe segue o padrão SQL.

Perguntas frequentes

- ❑ *Posso fazer tudo no backend e ignorar SP?*

Pode, mas vai perder **segurança**, **performance** e **consistência**.

Bancos modernos não são caixa de dados - eles processam dados.

- ❑ *Por que o PostgreSQL usa CALL, e não SELECT?*

CALL = procedimento (ação)

SELECT = função (retorna valor)

No PostgreSQL moderno, a sintaxe segue o padrão SQL.

- ❑ *Se eu não usar procedure, onde ficaria essa lógica?*

- ❑ aplicação (backend)
- ❑ código repetido
- ❑ menos controle

Resumindo

❑ Boas Práticas

Nome claro (sp_registrar_medicao)
Validar parâmetros
Documentar
Modularizar
Usar transação quando necessário

❑ Vantagens

Performance
Segurança
Reutilização
Consistência
Automação

❑ Desvantagens

Complexidade de manutenção
Dependência do SGBD
Controle de versão mais difícil

❑ Use SP quando

Regras de dado
Operações críticas
Rotinas repetitivas

SP = lógica **próxima do dado** -> profissionalismo e robustez

BANCO DE DADOS RELACIONAL

Atividade Prática Individual

O que veremos na próxima aula

- **Triggers**
- **Tópicos:**
 - O que são triggers
 - Para que servem
 - Tipos de triggers
 - Tipos de eventos



Referências Bibliográfica da Aula

Livros:

Elmasri & Navathe (2010). Sistemas de Banco de Dados.

Silberschatz et al. (2011). Sistemas de Banco de Dados.

Links úteis:

 [PostgreSQL Docs](#)

 [DBDiagram.io](#)

Bibliografia Básica

- ❑ DATE, C. J. **Introdução a sistemas de bancos de dados**. Rio de Janeiro, Elsevier: Campus, 2004.
- ❑ ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7 ed. São Paulo: Pearson, 2018.
- ❑ SILBERSCHATZ, A.; SUNDARSHAN, S.; KORTH, H. F. **Sistema de banco de dados**. Rio de Janeiro: Elsevier Brasil, 2016.

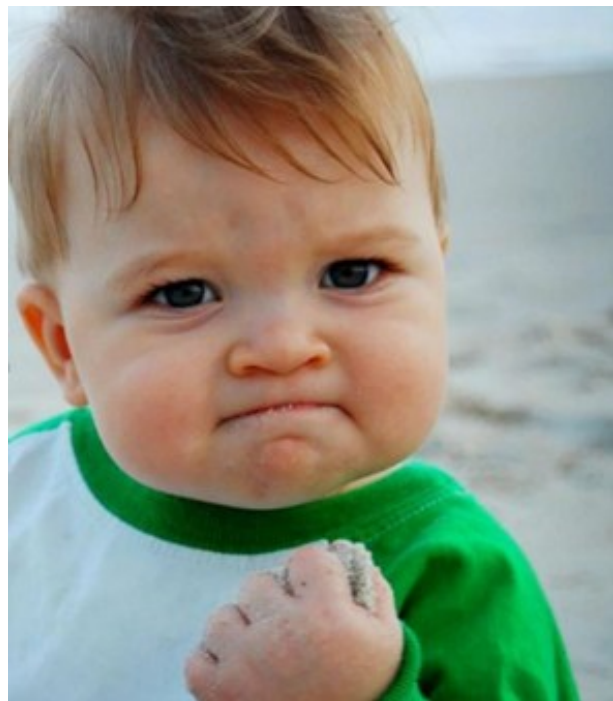
Bibliografia Complementar

- ❑ BEAULIEU, A. **Aprendendo SQL**. São Paulo: Novatec, 2010.
- ❑ GILLENSON, M. L. **Fundamentos de Sistemas de Gerência de Banco de Dados**. Rio de Janeiro: LTC, 2006.
- ❑ MACHADO, F. N. R. **Banco de Dados: Projeto e Implementação**. São Paulo: Érica, 2005.
- ❑ OTEY, M; OTEY, D. **Microsoft SQL Server 2005: Guia do Desenvolvedor**. Rio de Janeiro: Ciência Moderna, 2007.
- ❑ RAMAKRISHNAN, R.; GEHRKE, J. **Sistemas de Gerenciamento de Bancos de Dados**. 3 ed. Porto Alegre: Bookman, 2008.
- ❑ ROB, P; CORONEL, C. **Sistemas de Banco de Dados: Projeto, Implementação e Gerenciamento**. 8 ed. São Paulo: Cengage Learning, 2011.
- ❑ TEOREY, T; LIGHTSTONE, S; NADEAU, T. **Projeto e Modelagem de Bancos de Dados**. São Paulo: Campus, 2006.

Dúvidas?



Considerações Finais



**Professora:
Lucineide Pimenta**

Bom descanso à todos!

